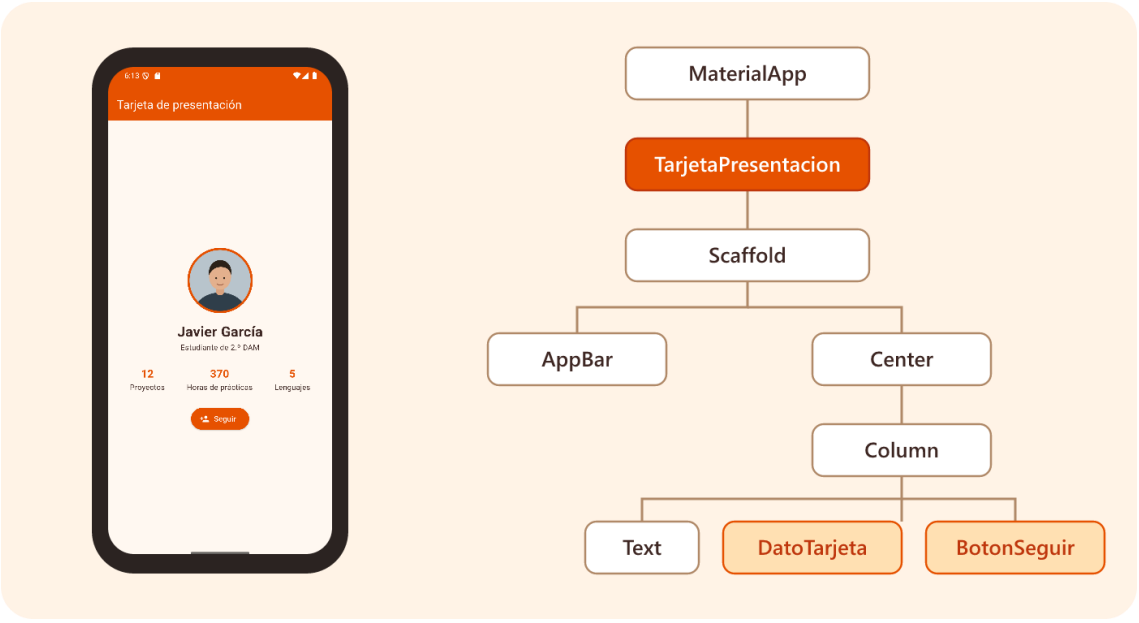
PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

2.º Desarrollo de Aplicaciones Multiplataforma

Widgets en Flutter

Trabajo de ejercicios: StatelessWidget, StatefulWidget y comunicación entre widgets



Javier García Cobos

Octubre de 2026

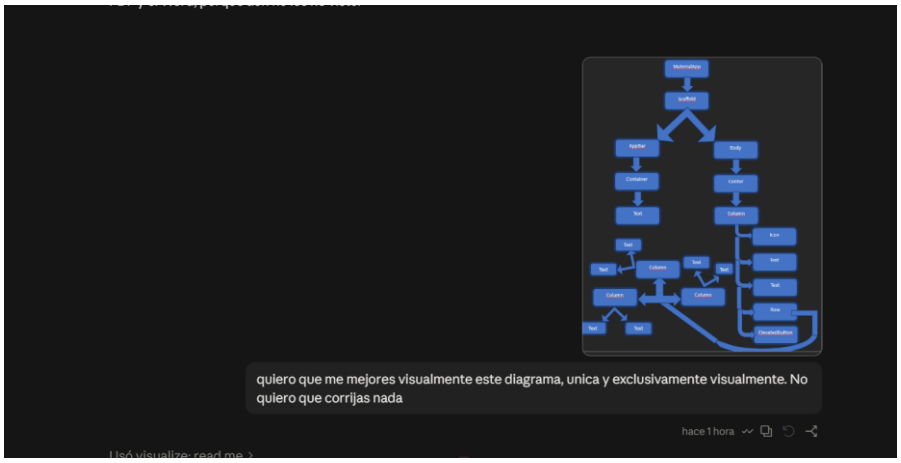
Herramienta de IA utilizada: Claude (Anthropic)

TRABAJO WIDGETS FLUTTER

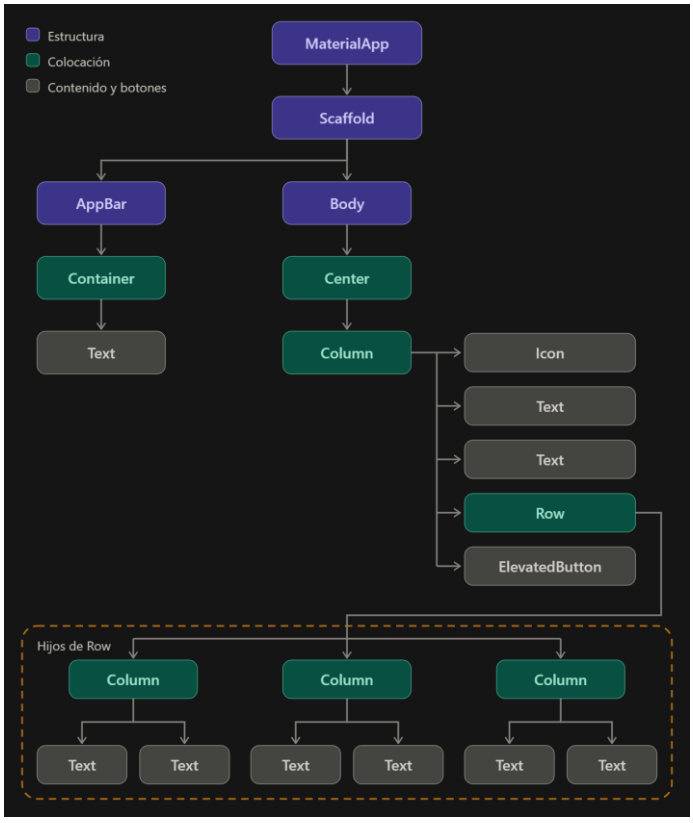
Ejercicio 1.

Árbol de widgets propio:

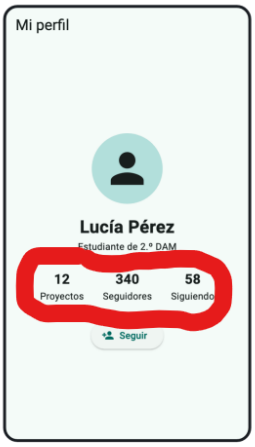
Hice mi propio árbol y se lo mande a Claude para que lo optimizara y quedara visualmente mejor.



Y este es el reultado:

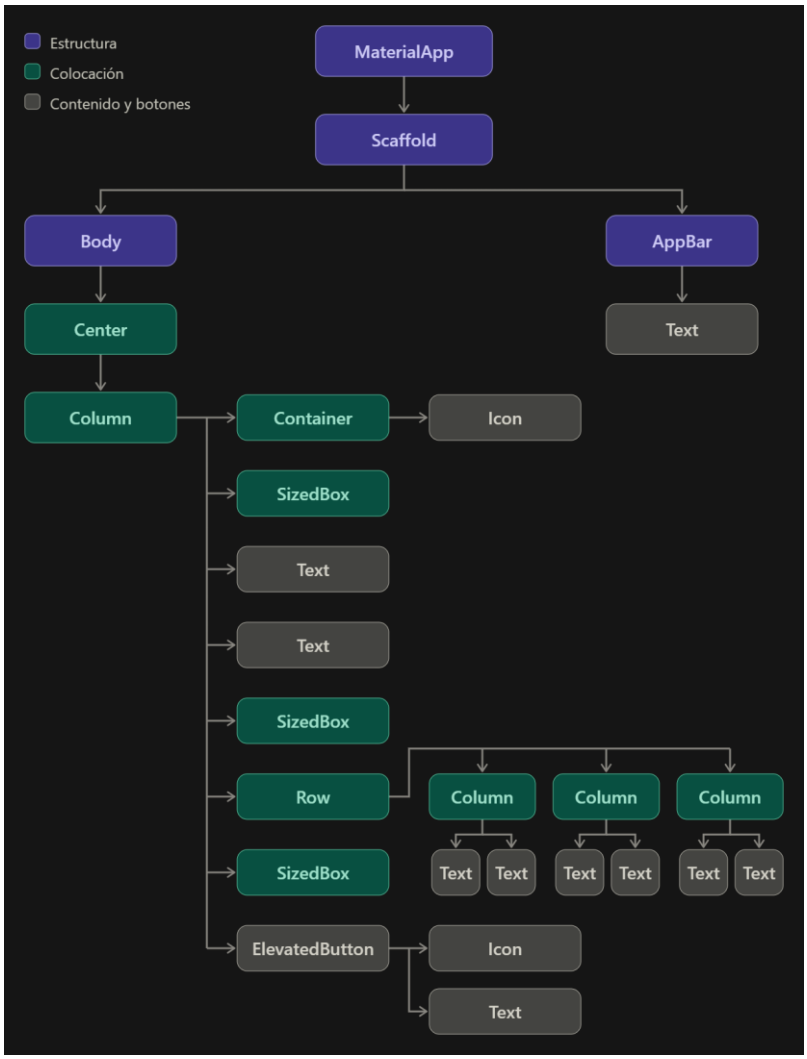


La parte redondeada corresponde a la repetición de los datos que están rodeados también en la siguiente imagen; únicamente cambia el contenido de la una a la otra.



Pantalla que debes analizar

Ahora le dije que me dijera el árbol de widgets que haría el en referencia a la imagen de la app, y me paso esto:



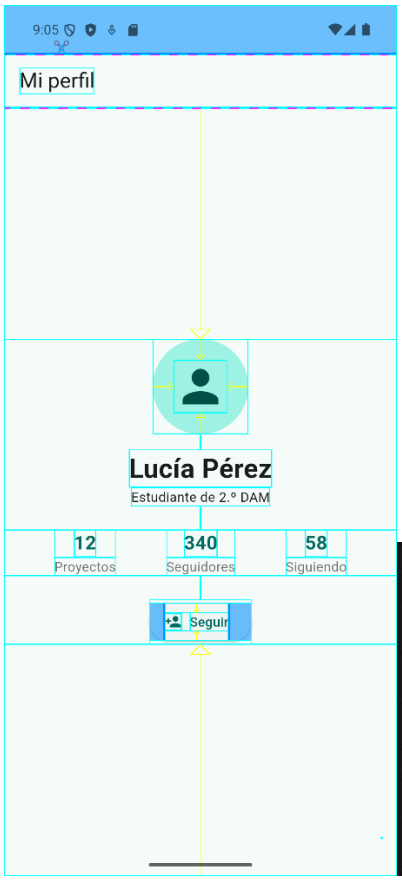
A continuación comparé ambos árboles viendo los widgets que coincidían y cuales no, deduciendo que mi árbol es más simple, faltándome algunos widgets de sidebox, algunos containers que no he puesto y otros que él no ha puesto.

Posición	Mi árbol	Árbol Claude	¿Coincide?
Raíz	MaterialApp → Scaffold	MaterialApp → Scaffold	✓ Sí
Hijos del Scaffold	AppBar y Body	AppBar y Body	✓ Sí
Dentro de AppBar	Container → Text	Text directo	✗ No – Yo tengo un Container de más
Body	Center → Column	Center → Column	✓ Sí
1.er hijo de la Column	Icon	Container → Icon	✗ No – Claude envuelve el icono en un Container
2.º hijo de la Column	Text	SizedBox	✗ No
3.er hijo de la Column	Text	Text	✓ Sí
4.º hijo de la Column	Row	Text	✗ No – En el de Claude el Row va después
5.º hijo de la Column	ElevatedButton	SizedBox	✗ No
6.º hijo de la Column	(no existe)	Row	✗ No
7.º hijo de la Column	(no existe)	SizedBox	✗ No
8.º hijo de la Column	(no existe)	ElevatedButton	✗ No
Dentro de Row	3 Column, cada una con 2 Text	3 Column, cada una con 2 Text	✓ Sí
Dentro de ElevatedButton	Nada (hoja)	Icon y Text	✗ No

Ahora adjunto la pantalla de la aplicación, en la que he cambiado un poco los coleres del StatelessWidget.



Ahora muestro la aplicación con el debugPaintSizeEnabled=true, y la tabla de comparaciones con el árbol final que realicé de la aplicación dada inicialmente.



Widget del árbol	Qué se ve en la captura	¿Coincide?
`AppBar` → `Text`	Banda superior con un recuadro ajustado a "Mi perfil"	✅
`Center` → `Column`	Las flechas amarillas arriba y abajo muestran que el `Center` centra la columna en vertical	✅
`Container` → `Icon`	Cuadrado de 100 dp con el círculo y, dentro, el recuadro del icono (flechas amarillas: centrado)	✅
3 `SizedBox`	Tres huecos verticales (línea cian vertical): avatar–nombre, subtítulo-fila y fila-botón	✅
2 `Text`	Recuadros ajustados a "Lucía Pérez" y a "Estudiante de 2.º DAM"	✅
`Row` → 3 `Column` → 6 `Text`	Una banda de ancho completo con tres bloques, cada uno con dos recuadros de texto	✅
`ElevatedButton` → `Icon` + `Text`	Recuadro del botón con el icono y "Seguir" dentro	✅

Ejercicio 2:

N.º	Descripción del widget	¿Stateless o Stateful?	¿Qué cambia? (o por qué nada cambia)
1	Un texto con el nombre de la aplicación en la barra superior.	Stateless	No recibe la interacción del usuario
2	Un interruptor para activar el modo oscuro de una pantalla.	Stateful	Recibe la interacción del usuario
3	Una tarjeta que muestra el nombre de un contacto que recibe de su padre.	Stateless	No recibe la interacción del usuario
4	Un contador de vasos de agua que suma uno al pulsar un botón.	Stateful	Recibe la interacción del usuario
5	Una estrella de «favorito» que se rellena al tocarla.	Stateful	Recibe la interacción del usuario
6	Un botón que recibe por parámetro la función que debe ejecutar al pulsarlo.	Stateful	Recibe la interacción del usuario
7	Un reloj que muestra la hora y se actualiza cada segundo.	Stateless	No recibe la interacción del usuario
8	Un campo de texto que muestra debajo lo que vas escribiendo.	Stateful	Recibe la interacción del usuario

Tabla del Ejercicio 2 completada por la IA (Claude):

N.º	Descripción del widget	¿Stateless o Stateful?	¿Qué cambia? (o por qué nada cambia)
1	Un texto con el nombre de la aplicación en la barra superior.	**Stateless**	Nada cambia: el título es fijo y siempre muestra el mismo texto.
2	Un interruptor para activar el modo oscuro de una pantalla.	**Stateful**	Cambia su valor (activado o desactivado) al pulsarlo; hay que guardarlo y redibujar con setState().
3	Una tarjeta que muestra el nombre de un contacto que recibe de su padre.	**Stateless**	Nada cambia dentro de ella: el nombre llega por el constructor y la tarjeta solo lo muestra.
4	Un contador de vasos de agua que suma uno al pulsar un botón.	**Stateful**	Cambia el número de vasos (una variable int) con cada pulsación y la pantalla se redibuja.
5	Una estrella de «favorito» que se rellena al tocarla.	**Stateful**	Cambia una variable bool (favorito sí/no) que decide si la estrella está llena o vacía.
6	Un botón que recibe por parámetro la función que debe ejecutar al pulsarlo.	**Stateless**	Nada cambia en el botón: recibe la función (VoidCallback) por el constructor y el estado lo lleva el padre.
7	Un reloj que muestra la hora y se actualiza cada segundo.	**Stateful**	Cambia la hora cada segundo; hay que guardarla en el estado y llamar a setState() al actualizarla.
8	Un campo de texto que muestra debajo lo que vas escribiendo.	**Stateful**	Cambia el texto escrito con cada letra; hay que guardarlo y redibujar el texto de debajo.

Tabla de desacuerdos entre mi clasificación y la de la IA:

N.º	Descripción del widget	Mi respuesta	Respuesta de la IA	¿Quién tiene razón y por qué?
6	Un botón que recibe por parámetro la función que debe ejecutar al pulsarlo.	**Stateful**	**Stateless**	La IA. El botón no guarda ningún dato que cambie: recibe la función por el constructor (VoidCallback) y solo la ejecuta al pulsarlo. Que el usuario lo pulse no cambia al botón; en todo caso cambiará el estado del widget padre. Por eso es StatelessWidget.
7	Un reloj que muestra la hora y se actualiza cada segundo.	**Stateless**	**Stateful**	La IA. La hora cambia sola cada segundo, sin que el usuario haga nada. El reloj debe guardar la hora en su estado y llamar a setState() cada vez que se actualiza (por ejemplo con un Timer), así que es StatefulWidget. El estado también puede cambiar con el tiempo, no solo con la interacción del usuario.

Punto 4. Programación de un widget sin estado y otro con estado

Capturas de la aplicación funcionando, en el emulador Pixel 7:



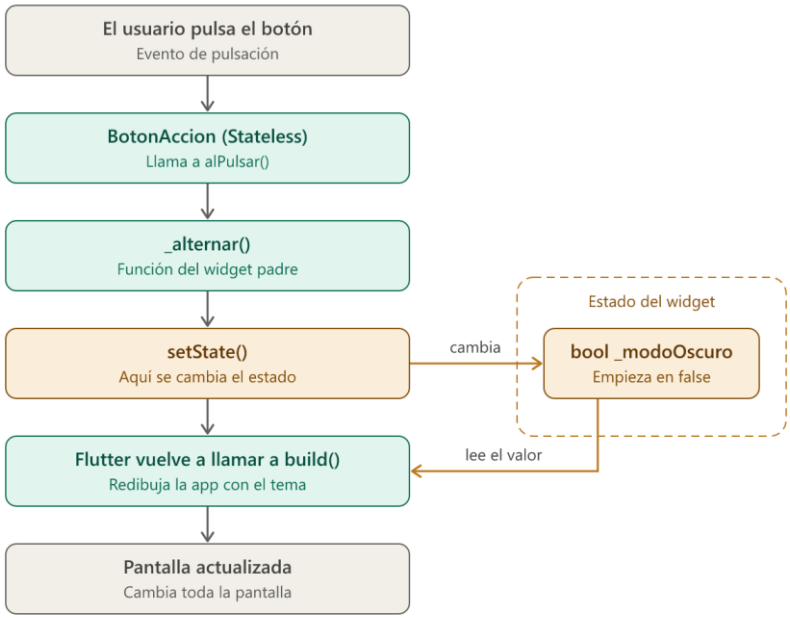
Modo claro (estado inicial)



Modo oscuro (tras pulsar «Activar»)

Punto 5. Esquema del estado del widget con estado

El estado de InterruptorModoOscuro es la variable bool _modoOscuro, que empieza en false. setState() se llama dentro de _alternar(), justo cuando el usuario pulsa el botón: ahí se invierte el valor de la variable. Después Flutter vuelve a ejecutar build(), que lee _modoOscuro y aplica el tema claro u oscuro a toda la pantalla.



Preguntas para analizar

Un widget puede mostrar datos que cambian con el tiempo y, aun así, ser StatelessWidget. ¿Cómo es posible? Busca un ejemplo en la tabla.

Porque el widget solo muestra los datos que recibe. Los que cambian están en el padre, que tiene estado, y este reconstruye al hijo con el valor nuevo. Ejemplo: la tarjeta de la fila 3, que muestra el nombre que le pasa su padre.

¿Qué inconveniente tendría hacer todos los widgets StatefulWidget «por si acaso»?

Más código y más complejidad sin necesidad, y se pierde claridad, porque no se sabe qué widgets cambian de verdad. Además hay más riesgo de errores con setState() y el estado queda repartido por toda la app.

Conclusión

Si un widget no cambia por sí mismo, es Stateless y recibe sus datos por el constructor; si cambia, es Stateful y guarda su estado en el State con setState(). Lo mejor es dejar el estado en un solo padre y que los hijos lo reciban y avisen con funciones.

Ejercicio 3:

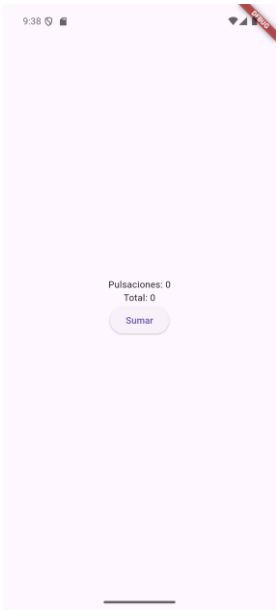
Punto 1.

La pantalla mostrará un texto que pone Pulsaciones, otro que pone Total y el botón que no funciona; por último, un texto que pone Sumar. Me he basado puro y duro en el código.

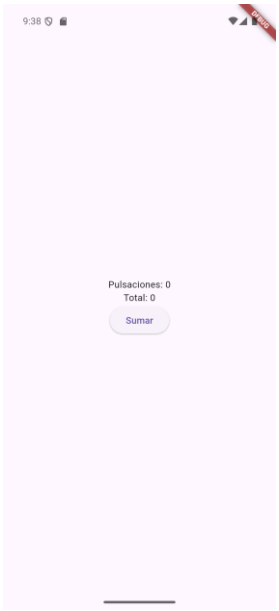
Punto 2.

Ejecuté el código en el emulador Pixel 7. Al abrir la aplicación aparece «Pulsaciones: 0», «Total: 0» y un botón con la palabra «Sumar». Después de pulsar «Sumar» tres veces, la pantalla sigue mostrando «Pulsaciones: 0» y «Total: 0»: no cambia nada.

¿Acerté? En parte. Acerté en que aparecen los textos «Pulsaciones» y «Total» y en que el botón no funciona. Me equivoqué en que «Sumar» no es un texto aparte, sino la etiqueta del propio botón. Además, mi predicción no decía qué números se mostrarían ni qué pasaría tras las tres pulsaciones, así que eso no lo puedo dar por acertado.



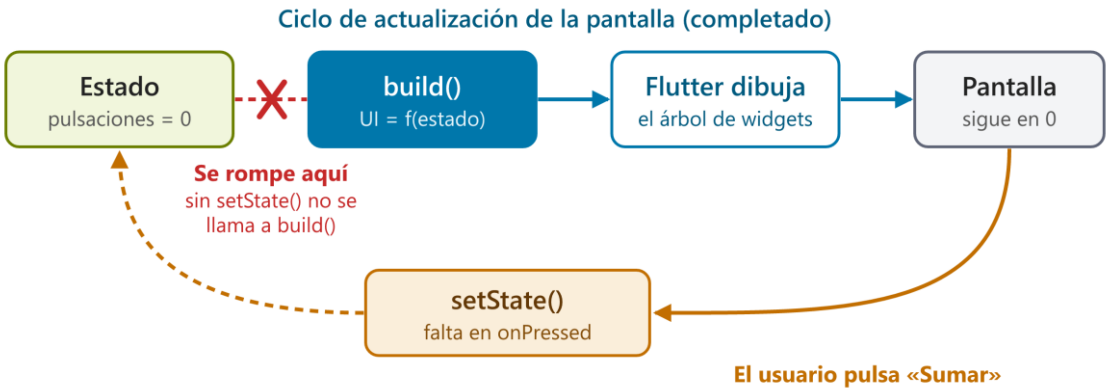
Al abrir la app



Tras pulsar «Sumar» tres veces

Punto 3.

Completé el ciclo de la figura:



Estado: «pulsaciones = 0». Al pulsar el botón su valor sube en memoria, pero la pantalla no lo muestra.

Hueco entre build() y Pantalla: Flutter dibuja en pantalla el árbol de widgets que devuelve build().

Hueco de abajo: setState(). Es lo que falta en el onPressed del botón.

Dónde se rompe el ciclo: entre el Estado y build(). Al pulsar, pulsaciones cambia, pero como no se llama a setState(), Flutter no sabe que ha cambiado y no vuelve a ejecutar build(). La pantalla se queda como estaba.

Además, total está declarada dentro de build(), así que se vuelve a crear con valor 0 cada vez que build() se ejecuta.

Punto 4.

Explicación de la IA (resumen):

1. No se llama a setState(). pulsaciones++ y total++ se ejecutan al pulsar, pero Flutter no se entera de que algo ha cambiado, así que no vuelve a ejecutar build() y la pantalla no se actualiza.

2. total está declarada dentro de build(). Es una variable local: se crea con valor 0 cada vez que build() se ejecuta, así que aunque la pantalla se

redibujara, Total mostraría siempre 0. El total++ del botón solo modifica esa copia y se pierde.

3. pulsaciones sí se acumula, porque es un atributo de la clase State. Si Flutter redibuja el widget por otro motivo, aparecería con el valor correcto, pero Total seguiría en 0.

Valoración:

¿Coincide con la mía? En parte. Yo había visto que el botón no funcionaba, pero no sabía por qué. La IA explica la causa: falta setState().

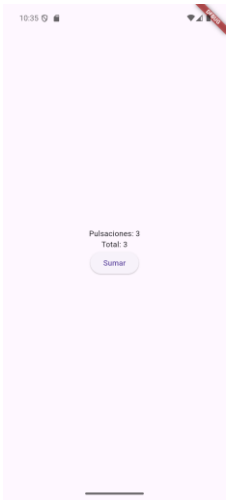
¿Ha encontrado todos los problemas? Sí, los dos: el setState() que falta y total declarada dentro de build(). Lo comprobé con un test: al forzar que Flutter redibuje el widget aparece «Pulsaciones: 3» y «Total: 0», como dice la IA.

¿Hay algo que no me haya convencido? No he encontrado nada que no cuadre: todo lo que explica se confirma al ejecutar el código.

Punto 5.

Corregí el código en dos sitios: llamé a setState() dentro del onPressed del botón y saqué la variable total de build(), declarándola como atributo de la clase State, junto a pulsaciones.

Después de pulsar «Sumar» tres veces, la pantalla muestra «Pulsaciones: 3» y «Total: 3»:



Punto 6.

Cambios que hice en el código:

Línea original	Línea nueva	Motivo
`int total = 0;` *(dentro del método build())*	`int total = 0;` *(como atributo de la clase State, debajo de pulsaciones)*	total era una variable local: se creaba con valor 0 cada vez que se ejecutaba build(), así que siempre mostraba 0. Como atributo del State conserva su valor.

`onPressed: () {`
`  pulsaciones++;`
`  total++;`
`},`
	`onPressed: () {`
	`  setState(() {`
	`  pulsaciones++;`
	`  total++;`
	`  });`
	`},`
		Sin setState(), Flutter no sabía que el estado había cambiado y no volvía a ejecutar build(), así que la pantalla no se actualizaba.

Preguntas para analizar

¿Por qué Flutter no actualiza la pantalla automáticamente cuando cambia el valor de una variable?

Porque Flutter no vigila las variables: solo vuelve a ejecutar build() cuando se le avisa con setState().

¿Qué diferencia hay entre declarar una variable dentro del método build() y declararla como atributo de la clase State?

Una variable de build() es local: se crea de nuevo, con su valor inicial, cada vez que build() se ejecuta, así que no guarda nada. Un atributo del State vive mientras exista el widget y conserva su valor. En el ejercicio, total (dentro de build()) siempre mostraba 0, y pulsaciones (atributo) sí acumulaba.

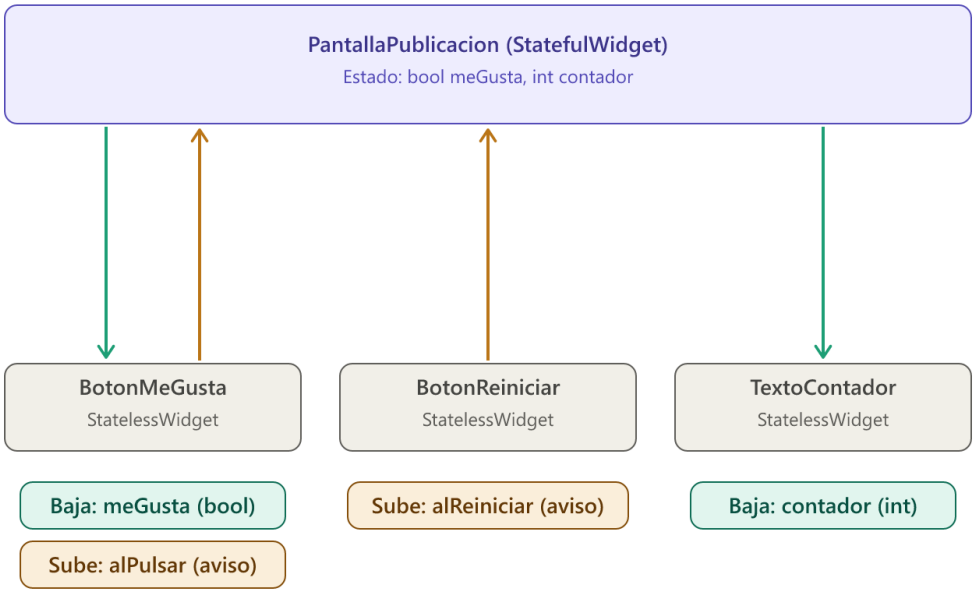
Conclusión

Para que la pantalla cambie, el dato debe guardarse en el State y modificarse dentro de setState(). Sin setState() la pantalla no se actualiza, y una variable local de build() no conserva su valor.

Ejercicio 4:

Punto 1.

El widget que guarda el estado es PantallaPublicacion, el único StatefulWidget, con las variables bool meGusta e int contador. Los tres hijos son StatelessWidget: BotonMeGusta, TextoContador y BotonReiniciar. Los datos bajan por el constructor y los avisos suben con funciones (VoidCallback); ningún hijo llama a setState().



Punto 2.

Herramienta de IA usada: Claude.

Prompt:

> *Quiero programar en Flutter una pantalla Publicación con un corazón, el texto "A N personas les gusta esto" y un botón Reiniciar. Mi diseño es este: un único StatefulWidget llamado PantallaPublicacion que guarda el estado (bool meGusta e int contador). Los hijos son tres StatelessWidget propios: BotonMeGusta (recibe meGusta y la función alPulsar), TextoContador (recibe contador) y BotonReiniciar (recibe la función alReiniciar). Al pulsar el corazón se marca o desmarca y el contador sube o baja en uno; Reiniciar pone el contador a cero y desmarca el corazón. Restricciones: solo un StatefulWidget en toda la*

pantalla; los hijos no pueden llamar a setState() ni modificar el estado; los atributos de los hijos deben ser final; los hijos avisan al padre con funciones recibidas por el constructor. Usa solo estos widgets: MaterialApp, Scaffold, AppBar, Center, Column, Row, Padding, SizedBox, Container, Text, Icon, ElevatedButton, OutlinedButton, TextButton e IconButton.

La IA me devolvió el código completo de la pantalla. En el siguiente punto lo reviso con la lista de comprobación.

Punto 3.

Revisé el código de la IA con la lista de comprobación del enunciado:

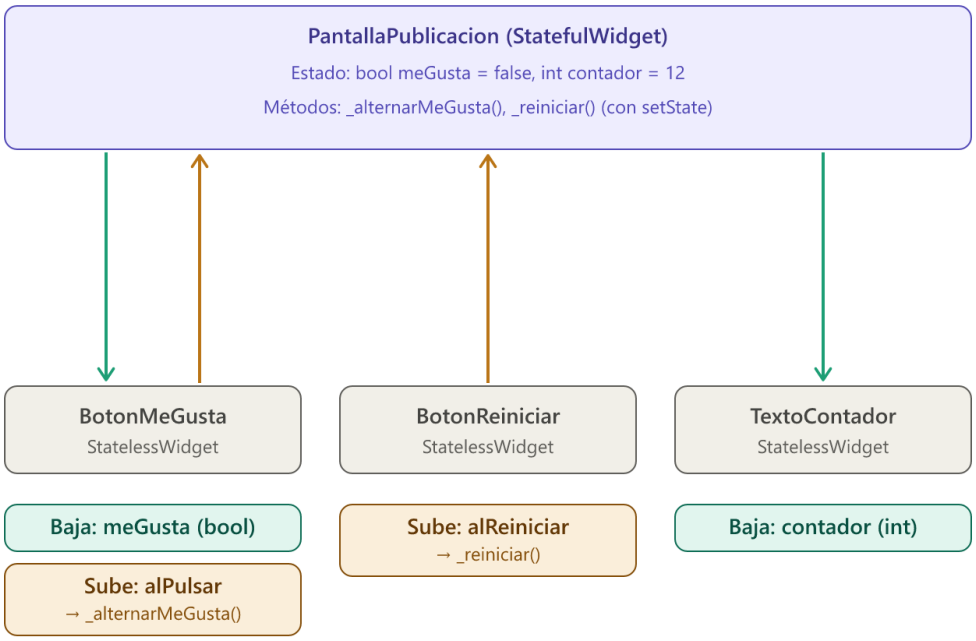
Comprobación	¿Se cumple?	Línea que lo demuestra (main.dart)
Hay un único StatefulWidget en toda la pantalla.	**Sí**	Línea 7: class PantallaPublicacion extends StatefulWidget. Es la única clase de este tipo en todo el archivo.
Los tres widgets hijos son StatelessWidget propios.	**Sí**	Líneas 64, 91 y 105: BotonMeGusta, TextoContador y BotonReiniciar extienden StatelessWidget.
Los atributos de los widgets hijos son final.	**Sí**	Líneas 65 y 66 (meGusta y alPulsar), 92 (contador) y 106 (alReiniciar): todos son final.
Los hijos avisan al padre mediante funciones recibidas por el constructor.	**Sí**	Líneas 66 y 106: alPulsar y alReiniciar son VoidCallback recibidos por el constructor. El padre se los pasa en las líneas 53 y 56.
setState() solo aparece en el widget padre.	**Sí**	Líneas 19 y 30, dentro de _PantallaPublicacionState (métodos _alternarMeGusta y _reiniciar). No aparece en ningún hijo.
Solo se usan widgets permitidos.	**Sí**	Líneas 38 a 58 y 76 a 114: Scaffold, AppBar, Padding, Column, Container, Center, Icon, SizedBox, Row, IconButton, Text y OutlinedButton. Todos están en la tabla de widgets permitidos.

Punto 4.

No hizo falta corregir nada: el código de la IA cumplía los seis puntos de la lista de comprobación. Cambios realizados: ninguno. Comprobé además que funciona: la cuenta arranca en 12, el corazón suma o resta uno y «Reiniciar» pone el contador a 0 y vacía el corazón.

Punto 5.

Este es el esquema final, con los nombres reales de los widgets:



Comparación con mi diseño inicial: la estructura es la misma, un padre con estado y tres hijos sin estado, con los mismos datos que bajan y los mismos avisos que suben. Lo único que cambia es que ahora aparecen los nombres reales de los avisos y de los métodos del padre a los que llaman (alPulsar llama a _alternarMeGusta() y alReiniciar llama a _reiniciar()), y los valores iniciales del estado (meGusta = false y contador = 12). La pantalla también tiene widgets fijos, la barra «Publicación» y la imagen, que no están en el esquema porque no tienen interacción.

Preguntas para analizar

¿Qué problema aparecería si el botón del corazón y el texto tuvieran cada uno su propio estado?

Cada uno tendría su propia copia de los datos y no se enterarían de los cambios del otro: el corazón podría marcarse y el texto seguir con el número anterior. Los datos se descuadrarían.

¿Qué ventaja tiene que los widgets hijos no sepan «qué pasa» cuando se les pulsa?

Los hijos son más simples y reutilizables, porque solo muestran lo que reciben y avisan. Toda la lógica está en un único sitio, el padre, así que es más fácil de mantener y de cambiar.

Conclusión

El estado debe vivir en un único widget padre: los datos bajan a los hijos por el constructor y los avisos suben con funciones. Así todo se mantiene sincronizado y los hijos son sencillos.

Ejercicio 5:

Punto 1.

Herramienta de IA usada: Claude.

Prompt inicial:

> *Quiero una app en Flutter que sea una tarjeta de presentación personal. Requisitos: 1) una barra superior con un título; 2) un icono grande dentro de un círculo de color; 3) mi nombre destacado y mi ciclo formativo debajo; 4) una fila con tres datos (proyectos, horas de prácticas y lenguajes) repartidos a lo ancho; 5) un botón que alterna entre Seguir y Siguiendo cada vez que se pulsa. Restricciones: usa solo estos widgets: MaterialApp, Scaffold, AppBar, Center, Column, Row, Padding, SizedBox, Container, Text, Icon, ElevatedButton, OutlinedButton, TextButton, IconButton, StatelessWidget y StatefulWidget; solo puede haber un StatefulWidget en toda la app; los widgets sin estado reciben los datos por el constructor y avisan con VoidCallback, y solo el StatefulWidget llama a setState(). Dame el código completo en un solo archivo main.dart.*

Por qué lo redacté así: puse los requisitos numerados para que la IA no se dejara ninguno. Incluí la lista de widgets permitidos y la restricción del único StatefulWidget porque son las reglas del enunciado y, si no, la IA suele proponer widgets más avanzados. Añadí cómo deben comunicarse los widgets (constructor y VoidCallback) para que el estado quede en un solo sitio. Y pedí el código completo en un solo archivo para poder ejecutarlo directamente.

Punto 2.

Audité la primera versión de la IA con esta tabla:

Requisito	¿Se cumple?	Evidencia (línea de main.dart o captura)
1. Una barra superior con un título.	Sí	Línea 46: AppBar(title: const Text('Tarjeta de presentación')). Se ve en las dos capturas.
2. Un icono grande dentro de un círculo de color.	Sí	Líneas 51 a 60: Container de 120 px con shape: BoxShape.circle y el color del tema, que contiene un Icon de tamaño 72.
3. Mi nombre destacado y mi ciclo formativo debajo.	Sí	Líneas 65 a 69: «Javier García» en negrita y tamaño 26 y, justo debajo, «Estudiante de 2.º DAM».
4. Una fila con tres datos repartidos a lo ancho.	Sí	Líneas 71 a 76: Row con spaceEvenly y tres DatoTarjeta (proyectos, horas de prácticas y lenguajes).
5. Un botón que alterna entre «Seguir» y «Siguiendo» cada vez que se pulsa.	Sí	Líneas 36 y 37: setState() invierte siguiendo. Línea 130: el texto cambia según siguiendo. Comprobado en las capturas 1_seguir y 2_siguiendo.
6. Solo widgets permitidos y un único StatefulWidget.	Sí	Único StatefulWidget en la línea 25 (TarjetaPresentacion); el resto son StatelessWidget (líneas 7, 89 y 115). Widgets usados: MaterialApp, Scaffold, AppBar, Center, Column, Row, Container, SizedBox, Text, Icon y ElevatedButton.

Nota: ElevatedButton.icon es un ElevatedButton con un icono, así que es un widget permitido.

Punto 3.

Widgets no permitidos: ninguno. El único que podía dar dudas es ElevatedButton.icon, que es un ElevatedButton con un icono delante del texto.

Líneas que no entendía: el botón de la clase BotonSeguir.

```
return ElevatedButton.icon(
  onPressed: alPulsar,
  icon: Icon(siguiendo ? Icons.check : Icons.person_add),
  label: Text(siguiendo ? 'Siguiendo' : 'Seguir'),
);
```

Lo que le pregunté a la IA:

> *¿Qué hace ElevatedButton.icon y qué significan el ? y los : dentro de icon y label? ¿Por qué BotonSeguir no es un StatefulWidget si su texto cambia?*

Lo que he aprendido:

ElevatedButton.icon: es otra forma de crear un ElevatedButton con icono. En vez de un solo child, recibe icon (el dibujo) y label (el texto).

siguiendo ? A : B: es el operador ternario. Significa «si siguiendo es verdadero, usa A; si no, usa B». Gracias a él, el mismo botón muestra «Seguir» o «Siguiendo» y el icono que le corresponde.

onPressed: alPulsar: la función se pasa sin paréntesis. No se ejecuta en ese momento, solo se indica cuál debe ejecutarse cuando se pulse el botón.

Por qué es Stateless: BotonSeguir no guarda nada. Recibe siguiendo y alPulsar de su padre. Al pulsarlo llama a alPulsar, el padre ejecuta _alternarSeguir() con setState() y vuelve a construir un BotonSeguir con el valor invertido.

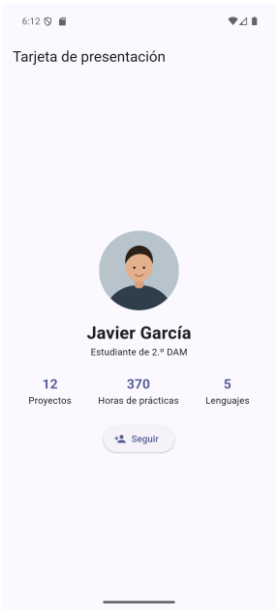
Punto 4.

Mejoré la versión en dos rondas. Guardé cada versión en la carpeta versiones del proyecto.

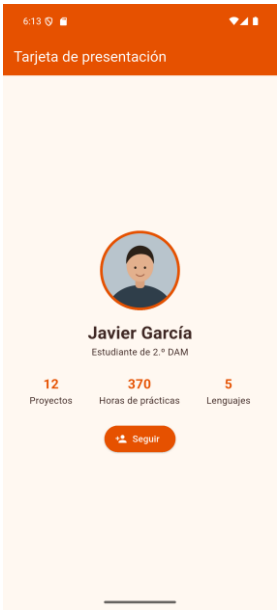
Ronda	Qué pedí	Qué cambió	¿Fue a mejor?
1	«Añade una foto de perfil en lugar del icono, usando solo los widgets permitidos.»	El icono del círculo se sustituye por la imagen assets/foto_perfil.png, dibujada como fondo del Container circular (DecorationImage con AssetImage dentro de BoxDecoration). No hace falta ningún widget nuevo, porque Image no está en la tabla. Hubo que declarar la imagen en pubspec.yaml.	Sí. La tarjeta es más personal y sigue cumpliendo las reglas. La foto es una imagen de muestra.

Ronda	Qué pedí	Qué cambió	¿Fue a mejor?
2	«Cambia la paleta de colores de la aplicación a naranja, con colores que combinen bien.»	Naranja intenso (#E65100) en la barra superior, el borde de la foto, los números y el botón «Seguir». Naranja suave (#FFE0B2) en el botón «Siguiendo». Fondo crema (#FFF8F1) y texto marrón oscuro (#3E2723). Todos son tonos cálidos, así que combinan.	Sí. Se ve más coherente y se distingue bien el estado del botón. El texto sigue siendo legible sobre el fondo crema.

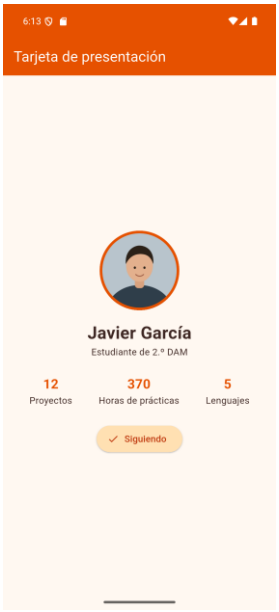
Capturas de las dos rondas:



Ronda 1: con foto



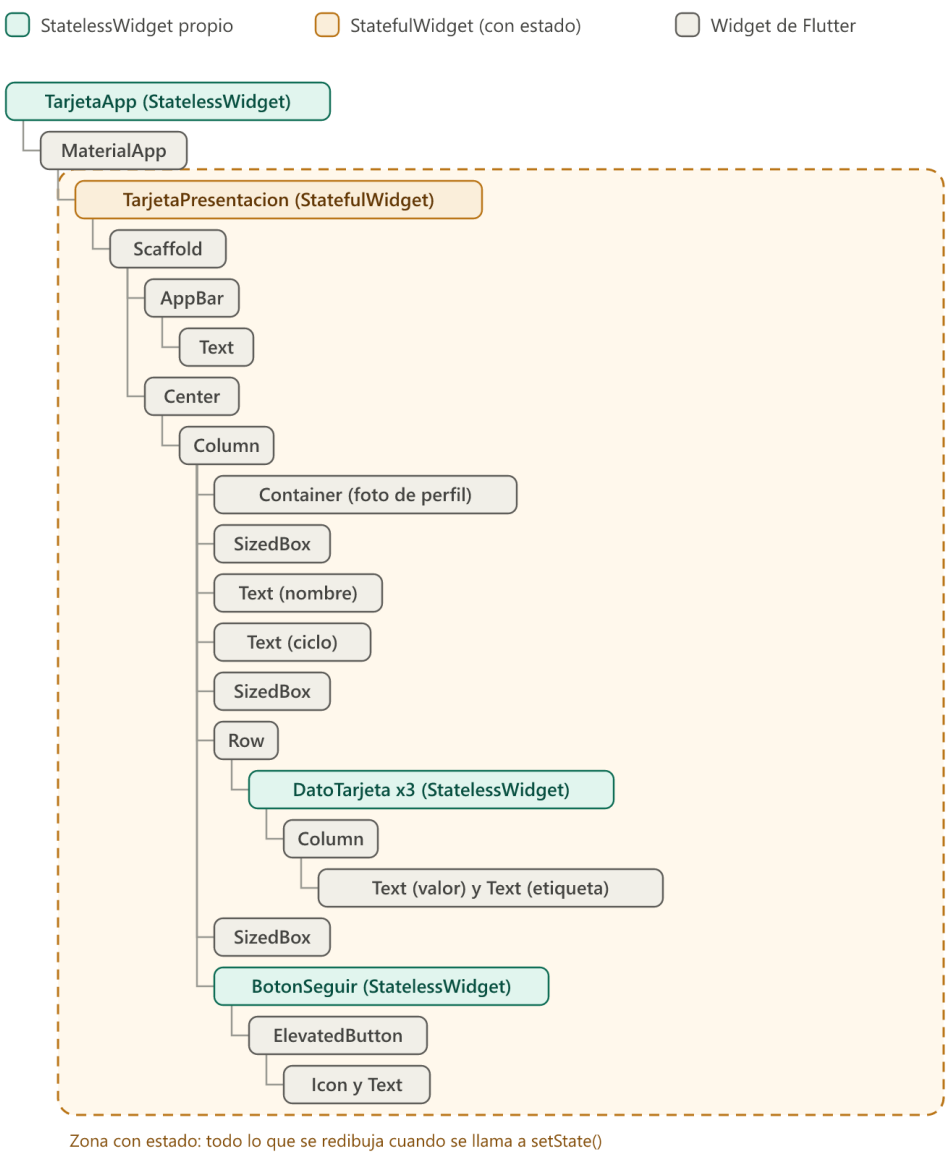
Ronda 2: «Seguir»



Ronda 2: «Siguiendo»

Punto 5.

Este es el árbol de widgets final. La parte con estado está marcada en naranja: es TarjetaPresentacion, el único StatefulWidget, y todo lo que construye, que se vuelve a dibujar cada vez que se llama a setState().



Preguntas para analizar

Di tres cosas que la IA hizo bien a la primera y tres que tuviste que corregir o pedir de nuevo.

Bien a la primera: cumplió los seis requisitos; usó solo widgets permitidos y un único StatefulWidget; y creó un widget propio para la parte repetida (DatoTarjeta) y otro para el botón, que avisa con VoidCallback.

Pedir de nuevo: la foto de perfil, que no estaba en el primer prompt; la paleta de colores naranja; y un aspecto distinto del botón en «Siguiendo», porque en la primera versión solo cambiaban el texto y el icono.

Calcula de forma aproximada qué parte del código final entiendes por completo (en porcentaje) y explica qué te falta para llegar al 100 %.

Entiendo aproximadamente un 85 %. Me falta comprender bien los detalles del tema (ThemeData y ColorScheme, con fromSeed y copyWith) y cómo Flutter carga las imágenes desde assets (pubspec.yaml y AssetImage).

¿Qué cambiarías de tu primer prompt si tuvieras que repetir el ejercicio?

Pediría desde el principio la foto de perfil y la paleta de colores, y pediría que cada parte del código llevara una explicación corta. También pediría que los textos de la tarjeta estuvieran en constantes fáciles de cambiar.

Conclusión

La IA acierta bien cuando los requisitos y las restricciones son claros, pero hay que revisar su código con una lista y entender cada línea. Pedir mejoras pequeñas, una por ronda, y comprobar cada una en el emulador funciona mejor que pedirlo todo de golpe.